ITE183 – Laboratory 2: WebSocket Server

Name: Niel Rey Licanda

Subject: ITE183 – Web Systems and Technologies

Laboratory: Laboratory 2: WebSocket Server

GitHub Repository: https://github.com/nrey23/ITE183_Lab2_Websocket

I. Objectives

This activity aimed to build an event-driven WebSocket server using Bonezegei Scripting Language v1.3.3, handle client connections and disconnections, and broadcast messages in real time to multiple connected browser clients.

II. Environment and Tools

Windows; Bonezegei Script Interpreter v1.3.3; Visual Studio Code; Git and GitHub; Chrome web browser; Bonezegei HTTP package. A local MinGW threading runtime (libwinpthread-1.dll) was also needed on this computer to load the HTTP library.

III. Implementation and Results

1. Installation verification

I verified that the Bonezegei interpreter installed on my computer was version 1.3.3, as required in the laboratory instructions.

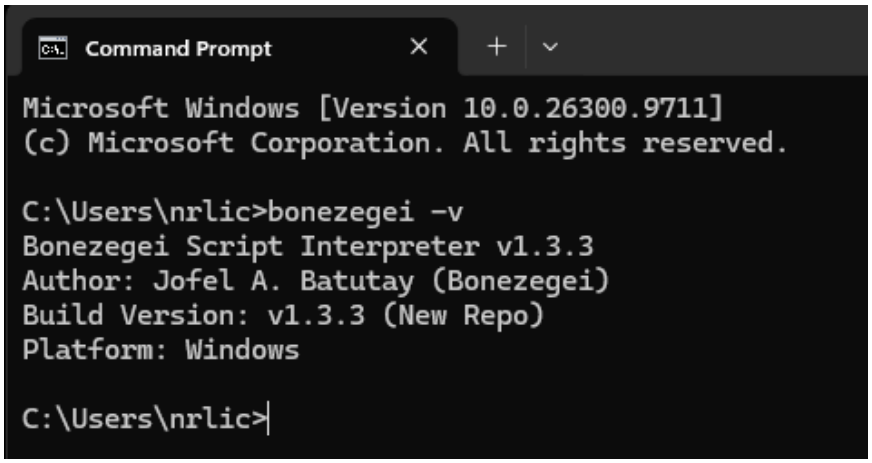


Figure 1. Bonezegei v1.3.3 verification in Command Prompt.

2. Installing the HTTP dependency

I ran bzg install inside the project folder. The terminal confirmed that the HTTP dependency was installed.

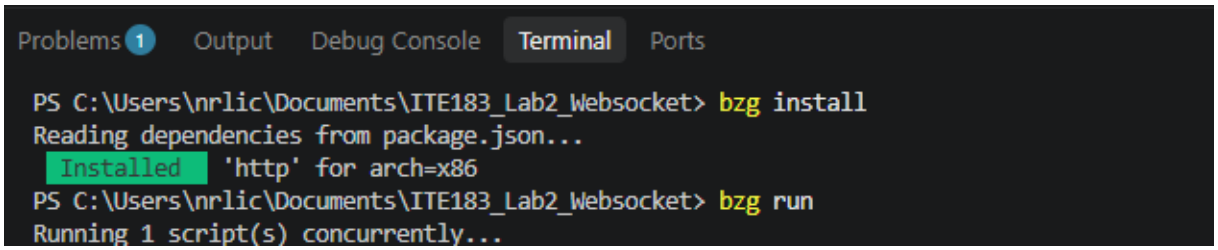


Figure 2. Successful HTTP dependency installation.

3. Starting the server

Initially, the HTTP DLL failed to load because the local Windows environment was missing libwinpthread-1.dll. After adding the verified dependency from the official MSYS2 package and retaining its license notice locally, I ran bzg run again. The server started without a visible error.

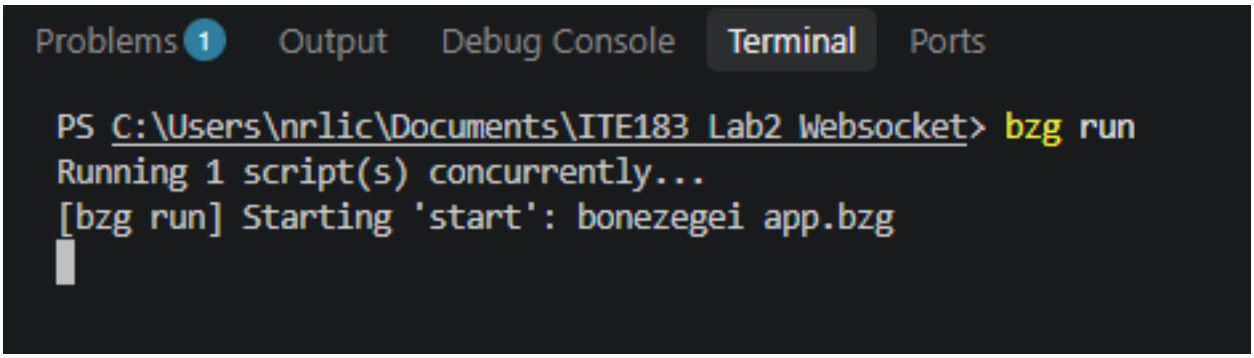


Figure 3. Successful server startup using bzg run.

4. Real-time multi-client broadcasting

I opened the WebSocket interface in two browser windows. Both showed a successful connection and handshake. A message sent by one client was received in both windows, confirming that server.sendAll() broadcast to the connected clients.

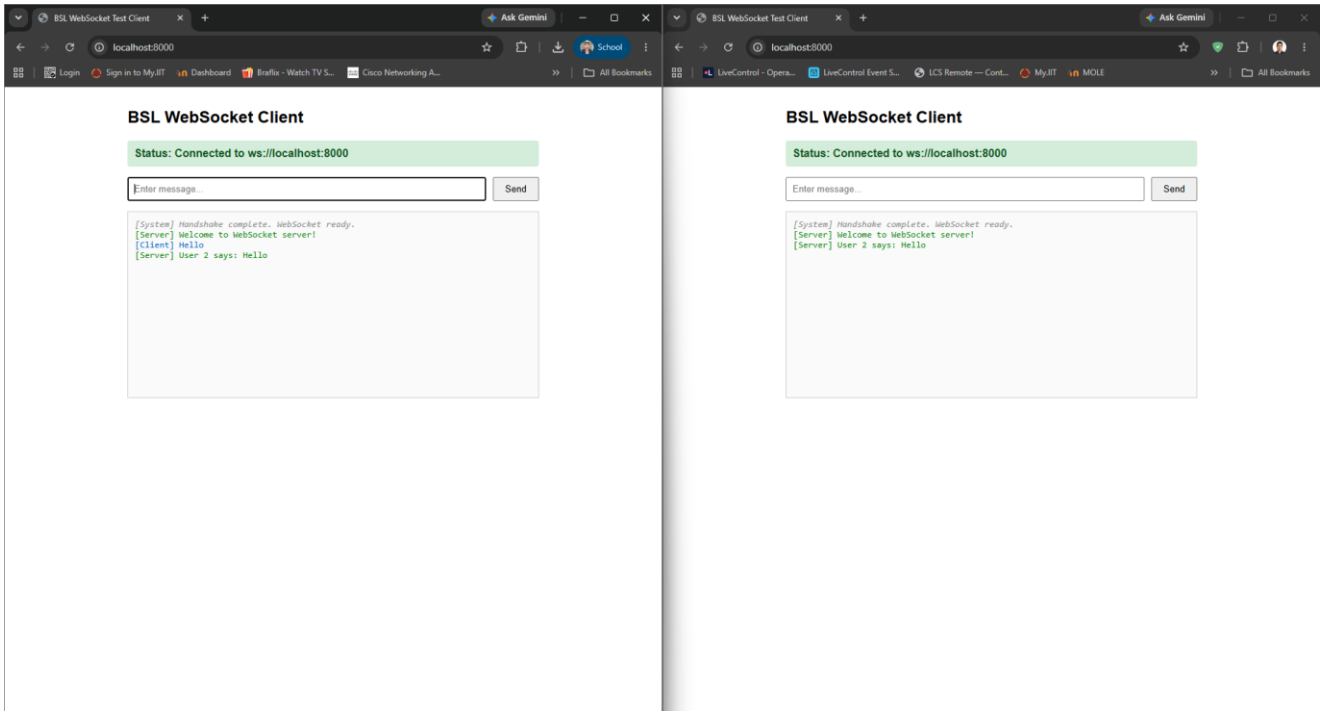


Figure 4. Two active WebSocket clients receiving the same broadcast.

5. Custom page for undefined routes

I accessed an undefined path on localhost:8000. The application displayed the custom 404 page from public/404.html. However, the current implementation returns HTTP status 200 rather than HTTP 404; this is a known limitation of the unchanged sample implementation.



Figure 5. Custom 404 page displayed for an undefined route.

6. GitHub repository

I committed and pushed the required source files to the public repository, including app.bzg, package.json, the public folder, README.md, MIT LICENSE, .gitattributes, and .gitignore. Locally installed libraries and the native DLL were excluded from version control.

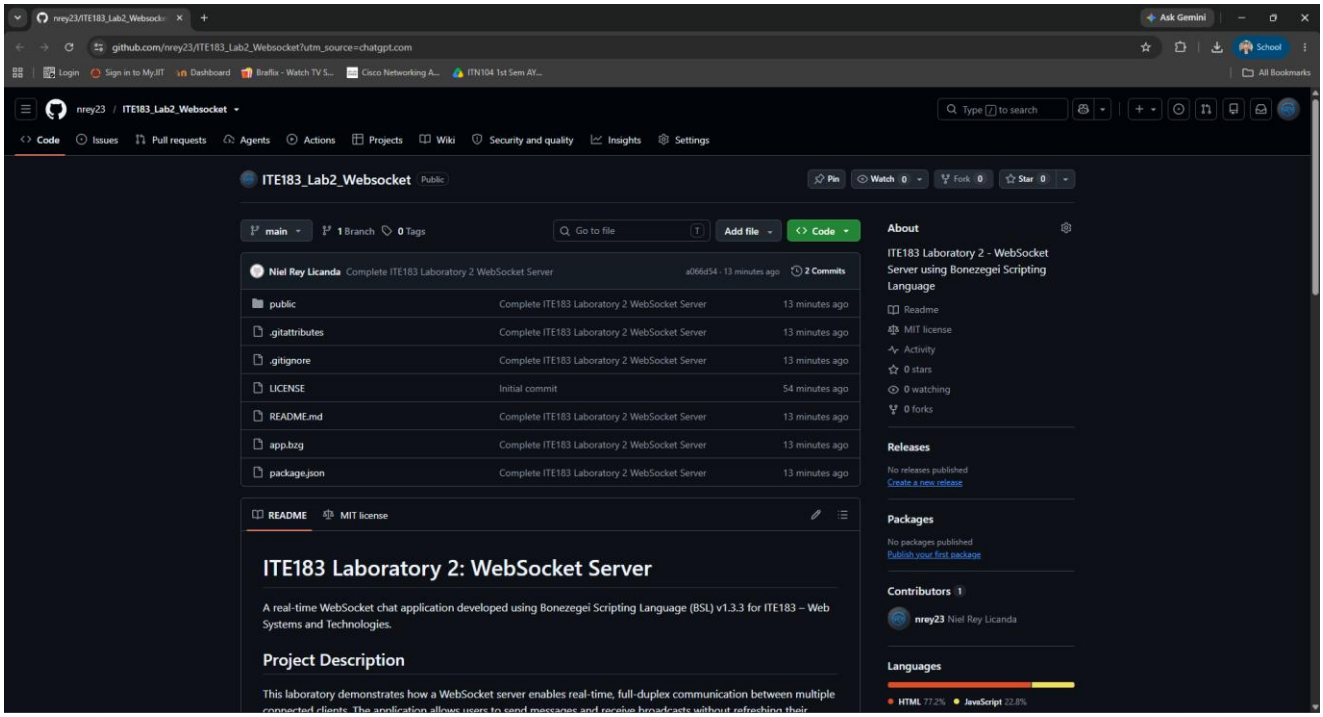


Figure 6. Public GitHub repository after the successful push.

IV. Testing Summary

Test performed	Observed result
Bonezegei version check	Passed – v1.3.3
bzg install	Passed – HTTP dependency installed
bzg run	Passed – server started
WebSocket connection	Passed – two clients connected and received welcome messages
Message broadcasting	Passed – message received in both windows
Undefined route	Custom 404 page displayed; HTTP response status remained 200
GitHub submission	Passed – files committed and pushed to main

V. Conclusion

In this laboratory, I learned how a WebSocket server keeps a connection open so clients can send and receive messages without refreshing the page. I also observed how connection callbacks and server.sendAll() allow real-time communication between browser windows. Troubleshooting the missing Windows runtime dependency helped me understand that server problems can come from the environment and not only from the source code. The application completed the required chat and custom-page demonstrations, with the HTTP 404 status-code limitation documented above.